BreakfastWise — Market Requirements Document

JJCET AI-PDLC Hackathon · 24–25 Oct 2026 · Owner: BA · Fact-check: QA · Condensed from `BreakfastWise_MRD_Final.md`

> **Markers: [A]** = assumption, not verified with a real hotel · **[S]** = built into our synthetic data · **[V]** = unverified claim · **[✓]** = public source (vendor-reported). All prototype data is **SYNTHETIC DEMO DATA — NOT REAL HOTEL DATA.**

1. Scope note

1. **Who:** the head chef of a mid-size hotel serving a South Indian breakfast buffet.
2. **Problem:** each night the chef guesses tomorrow's breakfast guests and how much idli, dosa, pongal, poori and sambar to make. Guessing high wastes food; guessing low means running out.
3. **What:** a Streamlit dashboard (**planned, not yet built**) that forecasts tomorrow's covers, then a prep quantity for each dish.
4. **How:** no machine learning. It uses capture rates (covers ÷ guests) averaged over the last 4 matching weekdays, with every step shown on screen.
5. **Timeline (24 Oct, 20:25 IST):** - **Complete:** MRD, PRD, backlog, synthetic dataset and the check script. - **Pending:** the app (S1–S9), README, test checklist, demo deck, AI log and backup recording. - About 3.5 h of building and 1 h of testing/demo prep remain (team's stated budget; stopping time unconfirmed). - Demos are 25 Oct at 10:00.

2. Problem and persona

Demand = guests in-house tomorrow × capture rate. - Occupancy and the breakfast-included split are known the night before **[A]**. - The capture rate varies by guest type and weekday **[A]**. - Events bring non-resident diners **[A]**. - Over-prep is waste; under-prep means unhappy guests and a rushed service **[A]**.

Users: - **Primary:** the head chef, who plans at 9 PM. - **Secondary:** the F&B manager, who watches waste and cost.

Persona — Chef Senthil Kumar (fictional) **[A]** - Age 41, 150-room business hotel in Tiruchirappalli, 18 years in kitchens. - **Today:** reads the occupancy printout and adds "a bit extra to be safe." - **Pain point 1:** "Mondays the idli trays come back half full; Saturdays we run out of poori at 8:30." - **Pain point 2:** wasted budget, complaints and stress during service. - **Workaround:** gut feel plus a buffer, never written down or checked.

3. Alternatives

Alternative	What we found	Gap we target (hypothesis)
Gut feel + buffer	Chef's experience **[A]**	Nothing recorded, so it can't be checked
Spreadsheet of covers	Manual history **[A]**	No next-day forecast or dish quantities
Hotel PMS (e.g. Oracle OPERA)	Supports meal packages; no public source found for a breakfast forecast **[V]**	A forecast step, if PMS tools lack one
Winnow Foresight	Item-level kitchen demand forecast from occupancy and history, built with Hilton chefs; vendor reports up to 69% less leftover weight in pilots **[✓]**	**Direct competitor.** Our possible edge is a formula the chef can check by hand, no subscription, and a South Indian menu. These are untested hypotheses
Winnow Vision, Leanpath	Camera and scale waste tracking with analytics dashboards **[✓]**	They measure waste afterwards; we target the prep decision before service

Sources: - Hospitality Net — Winnow Foresight - Hotel Dive — Winnow Vision - Leanpath - Oracle OPERA Cloud

4. Needs, targets and results

Need	PRD feature
N1 Tomorrow's covers	F1
N2 Dish quantities with a chef-set buffer	F2
N3 See how numbers were calculated	F1, F2
N4 Past prep vs. consumption	F3
N5 Adjust for what the data can't know	F4 (Should)

Targets are project goals, not guarantees. Synthetic results don't prove real-world accuracy.

Target	Threshold	Result so far (synthetic)
T1 Accuracy: last 14 days, no leakage	MAPE ≤ 10%	**4.20%** ✓
T2 Beats a single-rate baseline	Two-rate MAPE < baseline	**4.20% vs 4.92%**, a small gain ✓
T3 Planning time	< 60 s, 3 first-time runs	Pending (app not built)
T4 Waste % **and** shortage % per dish (no cross-dish or money total)	All 5 dishes	Script: simulated waste 9.7–11.1%, shortage 0.7–1.9%, recorded waste 17.4–21.1%. App panel pending

How T1 and T2 were measured: - **Script:** `docs/evidence/verify_mrd_metrics.py`. It is a standalone check, **not the app**. - **Window:** 11–24 Oct 2026, 14 of 14 days evaluated. - **Formula:** MAPE = (100/n) Σ |F − A| / A, with F rounded to whole covers. - **Zero actuals:** days where actual = 0 are skipped and counted; there were none. - **Events:** the uplift is leak-free. A past event day uses the mean of *earlier* event days, because no pre-breakfast estimates were recorded; the day's own event covers are only scored. There were no event days in the window. - **T4 ranges:** per dish, from the full table in the final MRD. Excluding stock-out days can bias recorded waste % and limits how representative it is. In this data it raises it (all days: 15.3–18.2%). - **Caveat:** the data was generated with the method's own structure **[S]**. Most of the accuracy comes from weekday averaging.

5. Key insight and data rules

Insight: the method is *designed* to separate guests with breakfast included from other guests. - In our data, included guests eat about 3× as often (88% vs 28% on weekdays) **[S]**. These are chosen inputs, not industry facts. - On this data, the split improves accuracy only slightly.

Required data: covers split by included and non-included guests. We have not confirmed that real hotels can provide this **[A][V]**.

Event covers: - **Event covers** = diners *not staying* in the hotel. - **A staying group** counts as residents. - Each cover is counted once: `covers_total = covers_included + covers_other + event_covers`. - Event covers are left out of the capture rates and added once as an uplift. - **Dataset check:** the identity holds on all 90 days, and event attendees are not in the guest counts. A "Tour group" label was corrected to "Non-resident group breakfast"; no numbers changed.

Fallbacks (specified in the PRD, **not implemented**):

Situation	Behaviour
Fewer than 4 matching weekdays	Use the days available and show a warning
No matching weekday	Show an error
A group has 0 guests in history	Show an error
A group has 0 guests tomorrow	Valid: that group adds 0 covers
Guest-split columns missing	Error naming the columns; **no single-rate fallback** (PRD Won't-Have)

6. Assumptions and limitations

#	Assumption	If wrong	Handling
A1	Guest groups differ in capture rate	The split adds little	Compare with the baseline (T2)
A2	Weekday patterns repeat	Lags for up to 4 weeks after a change	Chef override; dates shown
A3	Event diners are recorded separately	Double-counted or missed covers	Boundary rule; editable uplift
A4	Each guest's dish mix is stable	Dish quantities are off at weekends	Buffer slider
A5	All 5 dishes are served daily	Menu changes break the dish rates	Out of scope
A6	9 PM bookings match the morning	Late check-ins push the forecast too low; no-shows push it too high	Buffer; override
A7	Stock-out days understate demand	Dish rates come out too low	Those days are excluded
A8	Holidays and festivals behave like normal days	Large one-off errors	Override; holiday flag is a Could-Have

7. Project state and open issues

Project state: - **Exists:** the synthetic data generator, 3 CSVs (90 days), the docs and the check script. - **Doesn't exist yet:** app code, database or tests. - **Planned:** a single-page Streamlit app reading the CSVs, with no database, login or APIs.

Open issues: 1. Confirm the real stopping time tonight. 2. Build the app, then measure T3 and T4. 3. **Event-evaluation limitation:** no pre-breakfast event estimates were recorded, so the mean of earlier event days stands in (booking assumed known the night before). Only 3 evaluable event days, so the uplift is not validated. The app should save nightly estimates (not a Must-Have). 4. Add a backlog acceptance criterion for the missing-split error (PRD F1.8). 5. "Wedding party" is ambiguous: wedding guests with rooms should count as residents. 6. Positioning against Winnow Foresight. 7. PMS capabilities are unverified. 8. Real hotels' ability to provide the data is unconfirmed.